

Lab 1

Nicholas Storti

June 17, 2026

1 Output

```
Setting up the problem...0.000019 s
Vector size = 1000
Allocating device variables...0.160569 s
Copying data from host to device...0.000033 s
Launching kernel...0.008952 s
NumBlocks: 4
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.000157 s
Vector size = 10000
Allocating device variables...0.134905 s
Copying data from host to device...0.000044 s
Launching kernel...0.008912 s
NumBlocks: 40
Copying data from device to host...0.000032 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.001515 s
Vector size = 100000
Allocating device variables...0.136751 s
Copying data from host to device...0.000099 s
Launching kernel...0.008942 s
NumBlocks: 391
Copying data from device to host...0.000210 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.013612 s
Vector size = 1000000
Allocating device variables...0.138380 s
Copying data from host to device...0.000459 s
Launching kernel...0.008971 s
NumBlocks: 3907
Copying data from device to host...0.001236 s
Verifying results...TEST PASSED
```

Listed above are the outputs for vectors of size 1k, 10k, 100k, and 1M. All vector sizes passed the tests, which shows that the code is functioning properly. It can be seen from the output that the time for each

section does not change by the same amount as the vector size increases.

The "Setting up the problem" stage time increases roughly by a multiple of 10 as the vector size increases by 10. This is because this stage is where the input vectors are created. The input vectors are created using a for-loop which iterates over each element of the input vectors. The number of iterations is proportional to the input vector size, and the for-loop executes sequentially. Thus, the execution time is proportional to the number of for-loop iterations and the size of the input vectors. As the vector size increases, the number of for-loop iterations increases proportionally along with the execution time.

The "Allocating device variables" stage time stays roughly the same regardless of the vector size. This is because the memory is being allocated, but not written to. Allocating the memory without writing to it does not depend on the vector size.

Both the "Copying data from host to device" and "Copying data from device to host" stage times increase by roughly the same factor as the vector size increases. This is because copying the data from host to device, then copying the data back from device to host is a sequential task. Not all of the data can be copied at the same time. Because of this, as the vector size increases, it takes more time to copy all of the data over.

The "Launching kernel" stage time stays roughly the same regardless of the vector size. This is the stage where the kernel is called and the GPU performs the vector addition calculation. Because the GPU is performing the vector addition in parallel, the execution time increases very slowly relative to the vector size. Each element of the output vector is calculated on a separate core of the GPU in parallel with the other elements. The elements are added all at the same time each on a separate GPU core. Because of this, the execution time only increases by a small amount for large increases in vector size.

2 Main Code

Listing 1: main.cu

```
#include <stdio.h>
#include "support.h"
#include "kernel.cu"

int main(int argc, char**argv) {

    Timer timer;
    cudaError_t cuda_ret;

    // Initialize host variables

    printf("\nSetting up the problem..."); fflush(stdout);
    startTime(&timer);

    unsigned int n;
    if(argc == 1) {
        n = 10000;
    } else if(argc == 2) {
        n = atoi(argv[1]);
    } else {
        printf("\n----Invalid input parameters!"
            "\n----Usage: ./vecadd-----# Vector of size 10,000 is used"
            "\n----Usage: ./vecadd-<m>-----# Vector of size m is used"
            "\n");
        exit(0);
    }
```

```

}

float* A_h = (float*) malloc( sizeof(float)*n );
for (unsigned int i=0; i < n; i++) { A_h[i] = (rand()%100)/100.00; }

float* B_h = (float*) malloc( sizeof(float)*n );
for (unsigned int i=0; i < n; i++) { B_h[i] = (rand()%100)/100.00; }

float* C_h = (float*) malloc( sizeof(float)*n );

stopTime(&timer); printf("%f-s\n", elapsedTime(timer));
printf("----Vector-size-=%u\n", n);

// Allocate device variables -----

printf("Allocating-device-variables..."); fflush(stdout);
startTime(&timer);

//INSERT CODE HERE
float* A_d;
cuda_ret = cudaMalloc((void **) &A_d, sizeof(float)*n);
if(cuda_ret != cudaSuccess) printf("Unable-to-allocate-device-memory");

float* B_d;
cuda_ret = cudaMalloc((void **) &B_d, sizeof(float)*n);
if(cuda_ret != cudaSuccess) printf("Unable-to-allocate-device-memory");

float* C_d;
cuda_ret = cudaMalloc((void **) &C_d, sizeof(float)*n);
if(cuda_ret != cudaSuccess) printf("Unable-to-allocate-device-memory");

cudaDeviceSynchronize();
stopTime(&timer); printf("%f-s\n", elapsedTime(timer));

// Copy host variables to device -----

printf("Copying-data-from-host-to-device..."); fflush(stdout);
startTime(&timer);

//INSERT CODE HERE
cuda_ret = cudaMemcpy(A_d, A_h, sizeof(float)*n, cudaMemcpyHostToDevice);
if(cuda_ret != cudaSuccess) printf("Unable-to-copy-memory-to-device");

cuda_ret = cudaMemcpy(B_d, B_h, sizeof(float)*n, cudaMemcpyHostToDevice);
if(cuda_ret != cudaSuccess) printf("Unable-to-copy-memory-to-device");

cudaDeviceSynchronize();
stopTime(&timer); printf("%f-s\n", elapsedTime(timer));

// Launch kernel -----

printf("Launching-kernel..."); fflush(stdout);
startTime(&timer);

```

```

//INSERT CODE HERE
const unsigned int THREADS_PER_BLOCK = 256;
const unsigned int numBlocks = (n/THREADS_PER_BLOCK)+1;
dim3 gridDim(numBlocks, 1, 1), blockDim(THREADS_PER_BLOCK, 1, 1);

vecAddKernel<<<gridDim, blockDim>>>(A_d, B_d, C_d, n);
cuda_ret = cudaDeviceSynchronize();
if(cuda_ret != cudaSuccess) printf("Unable-to-launch-kernel");
stopTime(&timer); printf("%f-s\n", elapsedTime(timer));
printf("NumBlocks: %d\n", numBlocks);

// Copy device variables from host -----

printf("Copying-data-from-device-to-host..."); fflush(stdout);
startTime(&timer);

//INSERT CODE HERE
cuda_ret = cudaMemcpy(C_h, C_d, sizeof(float)*n, cudaMemcpyDeviceToHost);
if(cuda_ret != cudaSuccess) printf("Unable-to-copy-memory-from-device");

cudaDeviceSynchronize();
stopTime(&timer); printf("%f-s\n", elapsedTime(timer));

// Verify correctness -----

printf("Verifying-results..."); fflush(stdout);

verify(A_h, B_h, C_h, n);

// Free memory -----

free(A_h);
free(B_h);
free(C_h);

//INSERT CODE HERE
cudaFree(A_d);
cudaFree(B_d);
cudaFree(C_d);

return 0;
}

```

3 Kernel Code

Listing 2: kernel.cu

```

__global__ void vecAddKernel(float* A, float* B, float* C, int n) {

// Calculate global thread index based on the block and thread indices -----

//INSERT KERNEL CODE HERE

```

```
int i = blockIdx.x*blockDim.x + threadIdx.x;

// Use global index to determine which elements to read, add, and write —

//INSERT KERNEL CODE HERE
if (i < n) {
    C[i] = A[i] + B[i];
}
}
```